Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Claude Design for UX/UI

Course Code: C744

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 8.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
7.0	8 June 2026	Legacy Adobe XD slide-based courseware.	Tertiary Infotech Academy Pte Ltd
8.0	23 August 2026	Rebuilt for Claude Design; added principle-led content, eight aligned labs, current exports, accessibility, evaluation and handoff.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

How to Use This Guide

This guide carries the detailed procedures that are intentionally omitted from the concept-led slide deck. Use the module explanations before class activities, then follow each lab section click by click. Every lab has a matching self-contained folder and Sample-Code.pdf.

Boundary: The PowerPoint teaches concepts, frameworks and high-level workflows. This Learner Guide and the lab folders contain detailed steps, prompts, commands, code and acceptance checks.

Before You Start — What You Need

- A laptop with a current browser and permission to access Claude Design.
- An eligible Claude plan and, for Enterprise, administrator enablement where required.
- The downloaded labs folder and a local editor for HTML, CSS, JavaScript, Markdown and CSV files.
- A browser that can open local HTML files for Labs 4–7.
- No real personal, customer or confidential data in prompts, screenshots or shared prototypes.

Claude Design: https://claude.ai/design

Course LMS: https://lms-tms.tertiaryinfotech.com/

Public labs repository: https://github.com/tertiarycourses/C744-Claude-Design-for-UX-UI

Claude Design — Detailed Getting Started Guide

1. **Open Claude Design:** Go to claude.ai/design or use the Design entry in Claude Desktop. Sign in with your approved account.
2. **Create a project:** Choose New project. Use a project name that reflects one coherent design goal.
3. **Check the inherited system:** If your organisation has a design system, confirm the project has inherited it before generating.
4. **Add or import context:** Attach screenshots, documents, existing assets, design files or relevant codebase context. Use /design-sync from Claude Code when instructed for a connected system.
5. **Write the first prompt:** State the goal, audience, content, layout, required states, design-system constraints and acceptance criteria.
6. **Review the canvas:** Inspect the generated design on the right-side canvas. Check completeness before visual preference.
7. **Refine with chat:** Use chat for broad changes affecting layout, direction, hierarchy or multiple screens.
8. **Refine with comments:** Leave an inline comment on a specific element or region when the issue is local and benefits from context.
9. **Edit directly:** Correct deterministic text or properties directly on the canvas when no new generation is needed.
10. **Manage versions:** Save named checkpoints before broad changes and compare versions against the same criteria.
11. **Export or share:** Use Export to download ZIP, PDF, PPTX or standalone HTML, send to a supported tool, or hand off to Claude Code. Use organisation-scoped view, comment or edit sharing as appropriate.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

12. **Verify the downstream artifact:** Open the exported file or handoff bundle and confirm content, responsive views, component states, accessibility and version identity.

Claude Design and Claude Code Connection

The official help workflow supports a Claude Design MCP connection from Claude Code. Use it only with an approved environment and organisation account.

```
claude mcp add --scope user --transport http claude-design \
+  https://api.anthropic.com/v1/design/mcp
/design-login
/design-sync
```

Learning Outcomes

- LO1: Identify user needs, required functionality and information flows for an interface.
- LO2: Develop reusable interface components and states with Claude Design.
- LO3: Design, prototype and evaluate a responsive graphical user interface against defined metrics.
- LO4: Document design decisions, manage revisions, export deliverables and craft user guidance.

Criterion	Taught in	Assessed by

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Module 1 — Requirements and Principle-Led UI Design

From user needs and information flow to defensible interface decisions

Alignment: LO1

UI and UX: different lenses, one outcome

UX lens: The complete journey, task success and emotional experience.

UI lens: The visual and interactive layer users touch to complete the journey.

Shared ground: Research, flows, accessibility, testing and iteration connect both.

Outcome: A useful flow expressed through clear, predictable interface decisions.

Applied case: A booking page looks polished but hides the next available date below the fold. Repair the task flow first, then use hierarchy to make the next action obvious.

Evidence standard: Task completion and error evidence outrank aesthetic preference.

Source lens: Maze; UXPin

User-centricity begins with a job

Actor: Name the primary user and the context in which they act.

Job: State the outcome the user is trying to achieve, not the feature requested.

Barrier: Capture what makes the job slow, risky or confusing today.

Success: Define an observable result the interface should make easier.

Applied case: A team asks for a dashboard because competitors have one. Reframe the request around the decisions users must make and the signals they need.

Evidence standard: A strong brief connects business intent to a real user outcome.

Source lens: Microsoft; MockFlow

Design thinking with AI support

Empathise: Use interviews, observations and existing evidence; AI helps synthesize.

Define: Turn patterns into a focused problem statement and constraints.

Ideate: Generate divergent directions without confusing volume with quality.

Prototype + test: Make assumptions visible, test them, and iterate from evidence.

Applied case: Claude produces ten attractive concepts from a vague prompt. Anchor exploration in a research-backed problem statement and evaluation criteria.

Evidence standard: AI accelerates options; humans remain accountable for evidence and judgement.

Source lens: Figma; Maze

Research evidence: signal before synthesis

Behaviour: Analytics and observations reveal what people actually do.

Attitude: Interviews and surveys reveal expectations, language and motivations.

Context: Devices, interruptions, environment and accessibility needs shape use.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Triangulate: Combine sources before turning patterns into design requirements.

Applied case: Three interviewees request a feature that usage data shows almost nobody needs. Treat the request as a hypothesis and investigate the underlying task or barrier.

Evidence standard: A single source is a clue, not a conclusion.

Source lens: Contentsquare; Maze

Personas, segments and jobs-to-be-done

Persona: A research-backed behavioural archetype, not invented demographics.

Segment: A measurable group useful for product or service decisions.

Job story: When situation, I want motivation, so I can expected outcome.

Anti-bias check: Separate evidence from assumptions and decorative biography.

Applied case: A persona says 'busy professional' but provides no task, context or behaviour. Replace generic traits with a job story and evidence-linked constraints.

Evidence standard: Useful personas change a design decision.

Source lens: Maze; Microsoft

Information architecture: organise for recognition

Inventory: List content, actions, entities and states users need.

Grouping: Cluster items using user language and mental models.

Labels: Use specific nouns and verbs; avoid clever but ambiguous wording.

Validation: Card sorting and tree testing reveal findability problems early.

Applied case: A service portal groups content by internal department names. Reorganise around user goals and test whether users can predict where items live.

Evidence standard: Findability is measurable before high-fidelity design begins.

Source lens: Maze; MockFlow

User flows expose hidden interface states

Entry: Show where the user starts and what context arrives with them.

Happy path: Map the shortest valid route to the outcome.

Decisions: Name branch criteria and what the user needs to choose well.

Exceptions: Include empty, loading, error, permission and recovery states.

Applied case: A checkout flow documents payment success but not a declined card. Add failure, retry and saved-progress branches before designing screens.

Evidence standard: A complete flow maps states, not just pages.

Source lens: Microsoft; UXPin

Clarity: make purpose and next action obvious

Screen purpose: A clear heading says where the user is and why it matters.

Action labels: Use outcome-specific verbs such as 'Save draft' or 'Book slot'.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Visible priority: One primary action dominates; secondary actions remain available.

Plain feedback: Say what happened, what it means and what to do next.

Applied case: A form ends with two identical buttons labelled 'Continue'. Name outcomes, establish a primary action and clarify the consequence of each choice.

Evidence standard: If users must decode the interface, clarity has failed.

Source lens: Contentsquare; UXPin; MockFlow

Visual hierarchy: design the reading order

Scale: Larger elements signal importance when used sparingly.

Contrast: Value, colour and weight create emphasis and separation.

Position: Placement and whitespace determine what users encounter first.

Grouping: Proximity and alignment reveal relationships without explanation.

Applied case: Every card, heading and button on a dashboard has equal visual weight. Choose a reading order, strengthen the primary signal and quiet supporting content.

Evidence standard: A five-second glance should reveal purpose, status and next action.

Source lens: Figma; Visily

Consistency and familiarity build prediction

Internal: The same pattern looks and behaves the same across the product.

External: Conventions reduce relearning when they match user expectations.

Semantic: Colours, labels and icons keep the same meaning in every context.

Systemic: Tokens and components make consistency easier to maintain.

Applied case: The same icon opens search on one page and filters on another. Standardise meaning and encode the pattern in a reusable component.

Evidence standard: Consistency turns prior learning into speed and confidence.

Source lens: Microsoft; Maze; WeWeb

Simplicity and progressive disclosure

Essential first: Show what most users need to begin the task.

Advanced later: Reveal complexity when context or intent makes it useful.

Reduce decisions: Limit competing choices at each moment of the flow.

Preserve access: Do not hide frequently needed actions behind arbitrary layers.

Applied case: A settings page exposes forty controls in one uninterrupted list. Group by user goal and reveal advanced options only when requested.

Evidence standard: Simple means focused, not feature-poor.

Source lens: Figma; Dribbble; WeWeb

Accessibility is a design input

Perceivable: Contrast, text alternatives and scalable type support different senses.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Operable: Keyboard access, focus visibility and reachable targets support action.

Understandable: Plain language and predictable behaviour reduce confusion.

Robust: Semantic structure survives devices and assistive technology.

Applied case: A status uses only red and green dots with no labels. Add text or shape cues, verify contrast and include a visible focus state.

Evidence standard: Accessibility checks the interface under realistic variation.

Source lens: Figma; Microsoft; WeWeb

Feedback, error prevention and user control

Immediate feedback: Every action receives a visible, proportional response.

Prevent: Constraints and defaults block invalid actions before submission.

Recover: Errors explain cause, preserve input and offer a direct next step.

Control: Undo, cancel and review reduce the cost of mistakes.

Applied case: A destructive action succeeds silently and cannot be undone. Add confirmation where risk is high, show completion and provide recovery when possible.

Evidence standard: Good feedback keeps users oriented and in control.

Source lens: UXPin; Dribbble; Microsoft

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 1 — Research Brief and Requirements

Alignment: LO1 · Duration: 25 minutes

A neighbourhood wellness service needs a mobile experience for daily support and appointment booking.

Output: research-backed product brief, required-function list and measurable success criteria

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-01-research-brief-requirements`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 01 — Research Brief and Requirements`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-01-research-brief-requirements`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Separate evidence from assumptions** Complete `evidence-notes.csv`. Mark each input as evidence or assumption and write the design implication in the final column.
7	**Complete the requirements brief** Fill `brief-template.md` with the primary user, job, barriers, essential functions, constraints and measurable success. Ask Claude to identify gaps before generating a screen.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Names a primary user and job
- □ Separates evidence from assumptions
- □ Lists essential functions
- □ Defines observable success
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 2 — Information Flow and Principle Audit

Alignment: LO1 · Duration: 25 minutes

Map the wellness appointment task and inspect a flawed flow against clarity, hierarchy, consistency, accessibility and feedback.

Output: complete user flow with exception states and a prioritised UI audit

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-02-information-flow-principle-audit`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 02 — Information Flow and Principle Audit`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-02-information-flow-principle-audit`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Build the complete flow** Open `user-flow.mmd`. Confirm entry, happy path, availability decision, conflict recovery and confirmation. Add any required empty, loading or permission state.
7	**Audit against principles** Complete `audit-checklist.csv`. Every issue must name a principle, show observable evidence, state severity and describe how the correction will be verified.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Flow includes success and recovery
- □ Every issue names a principle
- □ Severity connects to task impact
- □ Recommendations are testable
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Module 2 — Claude Design Workflow and Design Systems

Context-rich prompting, systematic exploration and controlled refinement

Alignment: LO2

What Claude Design is—and is not

Design surface: Conversation and a live canvas support visual exploration.

Prototype engine: Outputs can be interactive and shared for feedback.

System-aware: Projects can inherit or import real design-system context.

Not autopilot: Research validity, accessibility and acceptance remain human duties.

Applied case: A stakeholder treats the first generated screen as a finished product. Position generation as a hypothesis and define the review evidence required before approval.

Evidence standard: The first generation is a starting point, not a sign-off.

Source lens: Anthropic

Access, interface and beta boundaries

Availability: Web and desktop access for supported Pro, Max, Team and Enterprise plans.

Workspace: Chat on the left; a working design canvas on the right.

Enterprise: The feature may require administrator enablement.

Beta limits: Expect evolving comments, large-repo and multi-editor behaviour.

Applied case: A class cannot see Claude Design even though accounts are active. Check plan eligibility, enterprise settings and web/desktop availability before the lab.

Evidence standard: Tool readiness is part of instructional readiness.

Source lens: Anthropic Help

A project is a context boundary

Purpose: One project holds a coherent design goal and its evolving decisions.

System: The project can inherit an organisation design system.

Inputs: Screenshots, documents, codebases and design files add concrete context.

Outputs: Versions, comments, exports and handoff remain linked to the work.

Applied case: Unrelated product concepts accumulate in one project and influence later generations. Separate goals into projects and keep context specific to the intended product.

Evidence standard: Good boundaries reduce accidental design drift.

Source lens: Anthropic Help

Import a design system before generating

Tokens: Colour, type, spacing, radius and elevation define visual decisions.

Components: Reusable structures encode interaction and accessibility patterns.

Guidelines: Voice, layout rules and do/don't criteria preserve intent.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Quality of source: Incomplete or messy inputs produce inconsistent outputs.

Applied case: A prompt says 'use our brand' but no components or tokens are attached. Import the actual source of truth and test a representative component before scaling.

Evidence standard: A design system is executable context, not a mood-board adjective.

Source lens: Anthropic Help; Interface Design

Prompt anatomy for interface work

Goal: Name the product outcome and the screen or flow to create.

Audience: Describe the primary user, context and accessibility needs.

Content: Provide real labels, entities, states and data priorities.

Constraints: Specify system, breakpoints, required states and acceptance checks.

Applied case: 'Make me a modern dashboard' produces a generic card grid. Add task, audience, content hierarchy, system rules and required states.

Evidence standard: Specific context beats stylistic adjectives.

Source lens: Anthropic Help

Direction without AI design slop

One personality: Choose a clear product character: precise, calm, warm or expressive.

One depth model: Use borders, tonal surfaces or shadows consistently.

One spacing system: A small scale prevents random gaps and component drift.

Explicit exclusions: Name patterns to avoid when they conflict with product intent.

Applied case: Every generated page uses gradient headlines, floating blobs and oversized rounded cards. Set a restrained direction and tie each visual treatment to product meaning.

Evidence standard: Polish comes from coherent decisions, not accumulated decoration.

Source lens: Interface Design; Developers Digest

Move from structure to fidelity

Low fidelity: Validate content, hierarchy, flow and screen coverage quickly.

Mid fidelity: Introduce components, realistic content and interaction states.

High fidelity: Apply brand, responsive behaviour and precise visual polish.

Prototype: Connect states so behaviour can be tested, not merely described.

Applied case: A team debates colour while the navigation model is unresolved. Return to structural wireframes, resolve the flow, then increase fidelity.

Evidence standard: Each fidelity level should answer a different question.

Source lens: Maze; Figma

Variants are hypotheses, not wallpaper

Diverge: Ask for alternatives that differ in information strategy, not just colour.

Name trade-offs: State what each direction optimises and what it sacrifices.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evaluate: Use the same success criteria for every variant.

Converge: Combine evidence-backed strengths rather than voting by taste.

Applied case: Three alternatives share the same layout and differ only in accent colour. Request distinct hierarchy and navigation strategies tied to the user goal.

Evidence standard: Useful variation changes the decision, not the decoration.

Source lens: Anthropic; Figma

Choose the right refinement channel

Chat: Broad changes affecting overall layout, tone or information strategy.

Inline comment: A local issue that benefits from contextual discussion.

Direct edit: Deterministic text or property corrections on the canvas.

Sliders: Controlled exploration of a repeated visual property.

Applied case: A broad request is repeated as dozens of tiny element comments. Use chat for the system-level change, then comments for remaining local exceptions.

Evidence standard: Match the communication channel to the scope of change.

Source lens: Anthropic Help

Versions turn iteration into evidence

Checkpoint: Save a named state before a significant direction change.

Compare: Review versions against the same task and criteria.

Explain: Record why the change was made and what evidence informed it.

Recover: Return to a known-good state when exploration degrades the design.

Applied case: A strong version is overwritten by successive experimental prompts. Create checkpoints and log the rationale before broad iterations.

Evidence standard: Version history supports judgement, not only undo.

Source lens: Anthropic Help

Responsive design is a set of priorities

Content priority: Decide what remains, collapses, moves or disappears by breakpoint.

Layout: Use fluid grids, min/max constraints and intentional stacking.

Interaction: Adjust navigation, touch targets and hover-dependent behaviour.

Validation: Test realistic content and states across widths.

Applied case: A desktop table is simply squeezed until labels become unreadable on mobile. Transform the information model: prioritise columns, support drill-down and keep actions reachable.

Evidence standard: Responsive is behavioural adaptation, not uniform shrinking.

Source lens: Microsoft; WeWeb

Tokens and components create a shared language

Token: A named decision such as space-3 or color-action-primary.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Component: A reusable structure with defined anatomy and behaviour.

Variant: A controlled expression such as primary, secondary or danger.

State: A condition such as hover, focus, disabled, loading or error.

Applied case: Every screen defines a slightly different blue and button height. Replace raw values with named tokens and a single button component contract.

Evidence standard: Systems reduce inconsistency by reducing repeated decisions.

Source lens: Figma; Interface Design

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 3 — Claude Design Project and Design System

Alignment: LO2 · Duration: 25 minutes

Create a Claude Design project and provide a compact design-system contract for the wellness product.

Output: project context pack with tokens, component rules and explicit design direction

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-03-claude-design-project-design-system`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 03 — Claude Design Project and Design System`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-03-claude-design-project-design-system`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Define the design direction** Open `project-context.md`. Confirm personality, depth model, spacing scale and explicit accessibility rules. Add it to the project context.
7	**Attach the system files** Add `tokens.css` and `component-contract.md`. Ask Claude to restate the token names and supported component states before building the test screen.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Uses named tokens
- □ Defines component variants
- □ Includes accessibility rules
- □ Test screen matches the system
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 4 — Prompted Wireframes and Variants

Alignment: LO2 · Duration: 25 minutes

Generate a four-screen wellness booking flow, then compare two structurally different directions.

Output: low-fidelity responsive wireframes and a documented variant decision

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-04-prompted-wireframes-variants`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 04 — Prompted Wireframes and Variants`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-04-prompted-wireframes-variants`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Load realistic content** Attach `content.json` so service names, durations and required states appear in the wireframes. Also open `wireframe.html` in a browser as the supplied structural reference.
7	**Generate a structural alternative** Ask for a second direction that changes hierarchy or navigation strategy, not only colour. Score both options in `decision-matrix.csv` using the same criteria.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.
9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ All required screens appear
- □ Real content replaces lorem ipsum
- □ Variants differ structurally
- □ Selection cites criteria
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Module 3 — Components, Prototypes and Evidence-Based Evaluation

Design realistic states, validate behaviour and iterate from observable results

Alignment: LO2 · LO3

Component anatomy defines responsibility

Container: Controls layout, grouping and responsive behaviour.

Content: Labels, icons and values communicate purpose and status.

Action: Interaction rules state what users can do and what follows.

Contract: Props, variants, states and constraints define safe reuse.

Applied case: A card component accepts unlimited actions and content shapes. Define supported anatomy and create separate patterns when responsibilities diverge.

Evidence standard: Reusable does not mean infinitely configurable.

Source lens: Figma; UXPin

States are part of the component

Default: The stable state communicates purpose and available action.

Interactive: Hover, pressed and focus states acknowledge user input.

Unavailable: Disabled and permission states explain why action is blocked.

System: Loading, success and error states reveal progress and outcome.

Applied case: A prototype contains only perfect default screens. Add the state matrix before visual polish and test transitions between states.

Evidence standard: A component without states is an illustration, not an interaction.

Source lens: Microsoft; WeWeb

Interaction design makes cause and effect visible

Trigger: Click, tap, type, drag, time or system event starts a transition.

Rule: Conditions determine whether the action is allowed and where it leads.

Response: Visual, content and navigation feedback confirms the effect.

Recovery: Undo, retry or correction protects users from cost and confusion.

Applied case: A button changes pages without preserving the user's entered data. Define state persistence and recovery as part of the interaction contract.

Evidence standard: Every interaction needs a comprehensible before, during and after.

Source lens: UXPin; Microsoft

Navigation should match the information model

Global: Persistent routes connect the product's major destinations.

Local: Tabs, subnavigation and filters organise a bounded context.

Contextual: Links and actions appear where their meaning is strongest.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Wayfinding: Current location, history and next options remain visible.

Applied case: A mobile app hides the current section and relies on a back button for all movement. Expose location and primary destinations without overwhelming the task view.

Evidence standard: Users should know where they are, what is nearby and how to recover.

Source lens: Maze; Dribbble

Forms are conversations with constraints

Ask less: Request only information needed for the current outcome.

Label clearly: Persistent labels beat placeholders that disappear.

Validate locally: Place specific guidance next to the field that needs attention.

Preserve progress: Do not erase correct input when one value fails.

Applied case: A long form reports 'Something went wrong' only after submission. Validate at useful moments, identify the field and explain how to correct it.

Evidence standard: Good forms reduce memory, rework and uncertainty.

Source lens: UXPin; Dribbble

Feedback and microcopy close the loop

Status: Tell users whether the system is working, waiting or complete.

Specific: Name the item and action rather than saying 'Success'.

Proportional: Match prominence and persistence to consequence.

Actionable: Errors and empty states offer a realistic next step.

Applied case: A data import runs for thirty seconds with no progress or cancellation. Show status, expected duration, safe cancellation and the final result.

Evidence standard: Feedback converts hidden system state into user confidence.

Source lens: Microsoft; MockFlow

Prototype the riskiest behaviour

Task risk: Prototype the path most important to user success.

Interaction risk: Test novel controls, dependencies and state transitions.

Content risk: Use realistic labels and edge-case data.

Technical risk: Represent latency, permissions and system boundaries honestly.

Applied case: A prototype animates the hero banner but ignores a complex approval flow. Move effort to the interaction whose failure would invalidate the design.

Evidence standard: Prototype uncertainty, not decoration.

Source lens: Maze; Anthropic

Define usability metrics before testing

Effectiveness: Task success, completion quality and critical error rate.

Efficiency: Time, steps and help required to reach the outcome.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Satisfaction: Confidence, ease and perceived effort after the task.

Accessibility: Keyboard, focus, contrast and assistive-technology checks.

Applied case: A team asks participants whether they 'like' a dashboard. Give a realistic task and measure success, time, errors and confidence.

Evidence standard: A metric must connect to the design's intended outcome.

Source lens: Maze; Contentsquare

Heuristic review is a structured expert lens

Status: The interface keeps users informed about what is happening.

Match: Language and concepts align with users' real-world understanding.

Control: Users can leave, undo and recover from unwanted states.

Consistency: Patterns follow product rules and familiar standards.

Applied case: Review comments say only 'this feels confusing'. Name the violated heuristic, show evidence and propose a testable correction.

Evidence standard: Heuristics turn opinion into an inspectable rationale.

Source lens: Maze; MockFlow

Accessibility review across the state matrix

Contrast: Check text, icons, controls and focus indicators in every state.

Keyboard: Order, reachability, traps and visible focus support non-pointer use.

Semantics: Names, roles, headings and error relationships convey structure.

Motion: Purposeful motion respects reduced-motion preferences.

Applied case: The default button passes contrast, but its disabled and focus states do not. Audit variants and states as a matrix rather than checking one screenshot.

Evidence standard: Accessibility defects often hide in transitions and exceptions.

Source lens: Figma; WeWeb

User testing observes behaviour, not performance

Task: Give a goal and context without teaching the path.

Observe: Record navigation, pauses, errors and spontaneous interpretation.

Probe: Ask neutral questions after behaviour occurs.

Synthesize: Group evidence by pattern, severity and affected outcome.

Applied case: A facilitator explains each feature while the participant attempts the task. Use neutral task wording and allow the interface to communicate for itself.

Evidence standard: Testing the participant invalidates evidence about the design.

Source lens: Maze

Iteration needs a change hypothesis

Evidence: Name the observed problem and who it affects.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Hypothesis: Predict how a specific change should improve the outcome.

Change: Alter the smallest coherent part that addresses the cause.

Retest: Use the same task and metrics to see whether the prediction holds.

Applied case: A redesign changes layout, copy and navigation after one failed task. Isolate the likely cause, make a bounded change and compare the same measure.

Evidence standard: Iteration becomes learning when the prediction is explicit.

Source lens: Contentsquare; Maze

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 5 — Reusable Components and States

Alignment: LO2 · Duration: 25 minutes

Turn the selected wireframe into a component system with accessible buttons, fields, cards and navigation.

Output: component sheet and coded reference with default, focus, loading, disabled and error states

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-05-reusable-components-states`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 05 — Reusable Components and States`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-05-reusable-components-states`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Inspect the coded reference** Open `index.html` in a browser. Tab through the controls and observe default, focus, busy, disabled and error states. Read `styles.css` and `states.js` in `Sample-Code.pdf`.
7	**Create the state matrix** Use `state-matrix.csv` to ensure every supported component shows the relevant variants and states on the Claude Design component sheet.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Components use tokens
- □ Focus is visible
- □ Loading/error states are distinct
- □ Reuse does not create drift
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 6 — Interactive Responsive Prototype

Alignment: LO3 · Duration: 25 minutes

Build and connect the high-fidelity wellness flow across desktop and mobile widths.

Output: interactive prototype covering booking success, conflict and recovery

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-06-interactive-responsive-prototype`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 06 — Interactive Responsive Prototype`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-06-interactive-responsive-prototype`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Run the starter prototype** Open `index.html` in a browser. Complete a normal booking, then choose 2:30 PM to trigger the unavailable-time state. Confirm the service selection is preserved.
7	**Connect the Claude Design prototype** Create or refine the high-fidelity screens and interactions to match the behaviour in `app.js`. Verify mobile and desktop layouts using `styles.css` as the contract.
8	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.
9	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

11	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

12	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Primary task is complete
- □ Conflict state preserves progress
- □ Mobile layout adapts
- □ All actions provide feedback
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Module 4 — Documentation, Collaboration and Handoff

Keep design intent, decisions and deliverables usable beyond the canvas

Alignment: LO4

Collaboration needs permission clarity

View: Stakeholders can inspect the current design without changing it.

Comment: Reviewers discuss decisions in context while authors retain control.

Edit: Collaborators can modify the design and should follow ownership rules.

Organisation scope: Sharing remains aligned with workplace access boundaries.

Applied case: A public edit link is shared for a confidential internal prototype. Match permission to the collaborator's role and review sensitive content before sharing.

Evidence standard: Convenience does not replace access governance.

Source lens: Anthropic Help

Choose export by downstream job

PDF: Fixed review, archive or presentation with stable layout.

PPTX: Editable presentation content for facilitated communication.

HTML / ZIP: Interactive handoff, local review or implementation starting point.

Tool transfer: Continue refinement in supported design or delivery tools.

Applied case: A developer receives only a flat screenshot of an interactive stateful prototype. Export an interactive or code-bearing format and include the design contract.

Evidence standard: The best format preserves what the next role needs to know.

Source lens: Anthropic Help

Claude Design and Claude Code handoff

Synchronise: Bring design-system context into the workflow with /design-sync.

Connect: The Claude Design MCP path supports design work from Claude Code.

Bundle: Handoff packages design intent, assets and structure together.

Verify: Implementation still needs visual, responsive and accessibility checks.

Applied case: Engineering rebuilds from a screenshot and loses component rules and states. Send the handoff bundle, design-system reference and acceptance criteria together.

Evidence standard: A shared source reduces translation loss but does not remove QA.

Source lens: Anthropic; Anthropic Help

A decision log explains why the UI changed

Context: Describe the user problem, constraint and affected flow.

Decision: State what changed and the component or screen impacted.

Rationale: Connect the change to research, testing or system rules.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Consequence: Record trade-offs, follow-up work and the new version.

Applied case: A button moved between versions with no explanation or test evidence. Add a concise decision entry tied to the observed navigation problem.

Evidence standard: Documentation preserves intent when people and tools change.

Source lens: Interface Design

User guides are task support, not feature inventory

Audience: Use the language and context of the people performing the task.

Goal: Organise guidance around outcomes, not product architecture.

Action: Give a short sequence with visible checkpoints and recovery.

Maintenance: Link guidance to versions and update it when the interface changes.

Applied case: A guide lists every screen but never explains how to complete a booking. Rewrite around the booking goal, including confirmation and common recovery paths.

Evidence standard: A guide succeeds when users complete work without interpretation.

Source lens: Microsoft; Maze

Case study: community wellness mobile flow

Need: Support users with varied confidence, vision and health literacy.

Flow: Onboarding → daily overview → appointment selection → confirmation.

System: Reusable navigation, cards, progress, support and error components.

Evidence: Task success, error recovery, focus order and language comprehension.

Applied case: An appointment slot becomes unavailable after the user selects it. Preserve the user's choices, explain the conflict and offer the nearest valid options.

Evidence standard: Resilient states matter more than the perfect happy path.

Source lens: Course case study

Case study: sustainable retail operations

Need: Help an operations manager spot exceptions and approve work quickly.

Hierarchy: Status cards lead to exception details and an approval queue.

States: Normal, empty, loading, error and focus behaviour are all represented.

Evidence: Time to detect risk, approval accuracy and keyboard completion.

Applied case: An approval card highlights amount but hides risk and requester context. Reorder information around the approval decision and show exception severity consistently.

Evidence standard: A dashboard earns its density by supporting real decisions.

Source lens: Course case study

Governance protects design quality at scale

Ownership: Name who approves tokens, components, content and accessibility rules.

Change control: Review impact before modifying widely reused patterns.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Privacy: Remove sensitive data from prompts, screenshots and shared prototypes.

Lifecycle: Deprecate, migrate and archive components with clear version guidance.

Applied case: A shared component changes silently and breaks dozens of screens. Require impact review, version notes and a migration path for breaking changes.

Evidence standard: Governance makes reuse safer, not slower.

Source lens: Microsoft; Anthropic Help

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 7 — Usability and Accessibility Evaluation

Alignment: LO3 · Duration: 25 minutes

Evaluate the prototype with a realistic task, heuristic review and accessibility checks, then perform one evidence-led iteration.

Output: test record, prioritised findings and before/after design checkpoint

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-07-usability-accessibility-evaluation`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 07 — Usability and Accessibility Evaluation`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-07-usability-accessibility-evaluation`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Define metrics before observation** Open `test-script.md`. Confirm task success, time, critical errors, help requests, confidence and keyboard completion before running the test.
7	**Record and prioritise evidence** Add observations to `results.csv`. Run the code in `accessibility-check.js` in the browser console as a starting check, then complete manual keyboard, focus, contrast and state review.
8	**Make one bounded iteration** Complete `iteration-log.md`: evidence, hypothesis, change, expected result, retest result and decision. Use the same task and metrics in the retest.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

10	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

11	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

12	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

13	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Metrics defined before testing
- □ Findings cite evidence
- □ Accessibility covers states
- □ Retest uses the same task
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 8 — Documentation, Export and Handoff

Alignment: LO4 · Duration: 25 minutes

Prepare the final prototype for stakeholder review, developer handoff and end-user guidance.

Output: versioned export bundle, decision log, handoff brief and task-based user guide

Before you start

- Use Claude Design on the web or desktop with an eligible account.
- Keep sensitive or personally identifiable data out of prompts, screenshots and exports.
- Use the self-contained files in `lab-08-documentation-export-handoff`.
- Read Sample-Code.pdf when you need a printable mirror of the supplied prompts or code.

Detailed step-by-step procedure

1	**Open Claude Design** Go to https://claude.ai/design. Sign in with the account approved for the class. If the Design option is not visible, verify the plan and ask the trainer to check organisation settings.
2	**Create the lab project** Create a new project named `Claude Design Lab 08 — Documentation, Export and Handoff`. Keep this lab separate from unrelated experiments so its context remains focused.
3	**Review the lab contract** Open `README.md`. Read the scenario, required output and acceptance checklist before generating anything.
4	**Attach the supplied context** Add the relevant files from `lab-08-documentation-export-handoff`. Do not attach confidential customer or learner data.
5	**Use the supplied prompt** Open `prompt.txt`, copy the complete prompt into the Claude Design chat, and submit it. Answer clarifying questions using the supplied evidence rather than inventing facts.
6	**Create the change record** Update `CHANGELOG.md` with the version, date, affected flow, change and evidence-backed rationale.
7	**Prepare the handoff** Complete `HANDOFF.md` with user goal, design system, required states, responsive behaviour and acceptance criteria. Include the correct export or Claude Code handoff bundle.
8	**Write task-based guidance** Complete `USER-GUIDE.md` around the booking outcome, confirmation checkpoint and unavailable-time recovery. Run every item in `export-checklist.md` before delivery.
9	**Inspect the first canvas** Check screen coverage, information hierarchy, labels, realistic content, required states, focus visibility and design-system consistency. Record any missing requirement before requesting changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10	**Refine at the right scope** Use chat for broad structure or direction changes. Use an inline comment for one local region. Edit deterministic labels directly on the canvas when no design judgement is required.

11	**Save a checkpoint** Create a named version checkpoint before the final change. Use a label that states the decision, for example `v1 — complete booking flow before usability iteration`.

12	**Verify the acceptance criteria** Work through every checkbox in the README. If a criterion is not observable in the design, revise the design or the evidence until it is clear.

13	**Export and record evidence** Use the Share or Export control appropriate to the lab. Save the requested output, share link or screenshot and record one design decision with its evidence.

Acceptance and evidence

- □ Change log explains rationale
- □ Export matches downstream need
- □ Guide supports a real task
- □ Handoff includes states and acceptance
- Final design/export or working file named in the lab output.
- Claude Design share link or named version checkpoint.
- Screenshot of the relevant final state.
- Decision note linking the change to evidence, principle or metric.

Troubleshooting

Symptom	Action
Design option not visible	Confirm plan eligibility and enterprise enablement; Claude Design is available on web and desktop.
Generic-looking output	Add real content, design-system files, audience, constraints and explicit patterns to avoid.
A comment seems missing	Open the comments view; beta comment persistence may hide inline placement.
Large project feels slow	Use a focused project or connect/synchronise the design system from Claude Code for large repositories.
The design regressed	Return to the named version checkpoint and reapply one bounded change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Quick Reference

Term	Meaning
Project	One coherent goal and its context, versions, comments and outputs.
Chat	Broad layout, hierarchy, direction or multi-screen changes.
Inline comment	A local issue that needs contextual discussion.
Direct edit	Deterministic text/property change on the canvas.
Version checkpoint	A named state saved before a significant change.
Export	ZIP, PDF, PPTX, standalone HTML, supported tool transfer or Claude Code handoff.
Evidence	Observed behaviour, metric, heuristic, accessibility finding or research source.

Sources and Further Reading

Figma — Seven essential UI design principles: https://www.figma.com/resource-library/ui-design-principles/

Microsoft — Key UI/UX design principles: https://learn.microsoft.com/en-us/dynamics365/guidance/develop/ui-ux-design-principles

UXPin — Essential UI design principles: https://www.uxpin.com/studio/blog/ui-design-principles/

Contentsquare — Core UI design principles: https://contentsquare.com/guides/ui-design/principles/

MockFlow — Design principles for intuitive UI: https://mockflow.com/resources/ui-design-principles

MockFlow — UI principles for usability: https://mockflow.com/blog/ui-design-principles

WeWeb — UI design principles: https://www.weweb.io/blog/ui-design-principles

Visily — Visual design principles: https://www.visily.ai/blog/visual-design-principles

Dribbble — Fundamental UI design principles: https://dribbble.com/resources/education/ui-design-principles

Maze — Key principles of UI design: https://maze.co/collections/ux-ui-design/ui-design-principles/

GeeksforGeeks — Principles of UI/UX design: https://www.geeksforgeeks.org/techtips/principles-of-ui-ux-design/

Duck.Design — Key UI design principles: https://duck.design/key-ui-design-principles/

Anthropic — Introducing Claude Design: https://www.anthropic.com/news/claude-design-anthropic-labs

Anthropic Help — Get started with Claude Design: https://support.claude.com/en/articles/14604416-get-started-with-claude-design

Developers Digest — Claude Design developer guide: https://www.developersdigest.tech/blog/claude-design-developer-guide

Interface Design — systematic UI patterns: https://github.com/Dammyjay93/interface-design

Support

For course support: enquiry@tertiaryinfotech.com · +65 6100 0613 · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg